Day 78: The Transformer Encoder Block, From Scratch

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 12, Day 2 | Complete study guide

Watch: Encoder Architecture in Transformers, Step by Step Guide (Learn With Jay) | PostLN, PreLN and ResiDual Transformers (Machine Learning Studio)

Script: `day78_encoder_block.py` · every number in this guide is copied by program from `day78_run_output.txt`, the real output of that script.

The Transformer wraps attention in a block: attention, a residual connection and layer normalisation, then a small feed-forward network applied to every position, with another residual and normalisation. Built from scratch, one block tags Week 9's sentences at 0.8510, a little above Day 77's single attention layer (0.8487). Of its parts, the feed-forward network matters most here. Untrained, LayerNorm alone keeps the gradient alive through 8 blocks. And stacking more blocks did not help: on this data, with plain SGD, no deeper stack beat one block.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

An **encoder block** has two halves, each with the same pattern, called **add & norm**: the half's input is added back to its output (the **residual**), and the sum is normalised (**LayerNorm**: each position's numbers to mean 0 and spread 1, then a learned scale and shift). The first half is Day 76's multi-head attention; the second is a **feed-forward network** (FFN), the same two-layer network applied to every position on its own.

REAL-WORLD SCENARIO

Editing a document in rounds: each editor suggests changes on top of the current text rather than rewriting it (the residual); between rounds the formatting is reset to a house style (the normalisation).

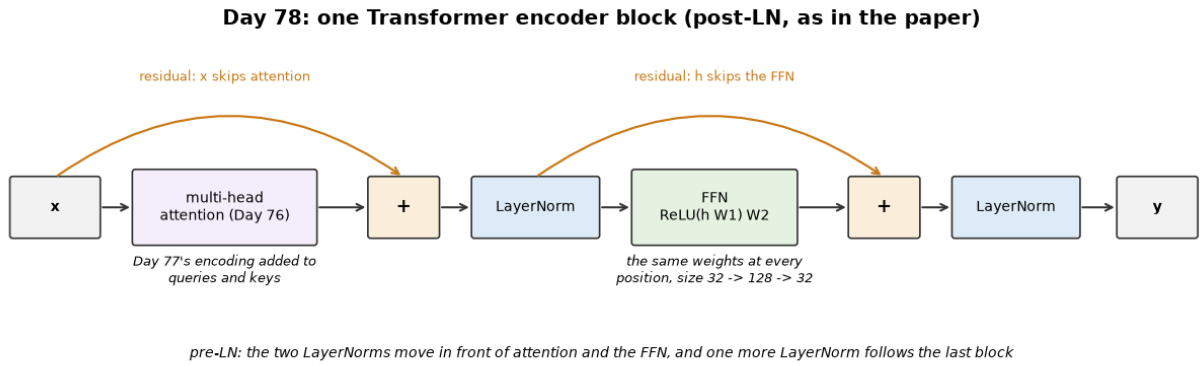
MATHEMATICS

post-LN (the paper): $h = LN(x + Attn(x)), \quad y = LN(h + FFN(h))$

pre-LN (most later models): $h = x + Attn(LN(x)), \quad y = h + FFN(LN(h))$, and one LN after the last block

$LN(x) = (x - \mu) / \sqrt{(\sigma^2 + \varepsilon)} \cdot g + b, \quad FFN(h) = ReLU(h\,W_1 + c_1)\,W_2 + c_2$

SYSTEM DIAGRAM



One post-LN encoder block.

Today's part for Day 83

`day78_exercise.py` is one post-LN block on its own, forward and backward, gradient-checked: Week 12's second part.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. Layer normalisation by hand

DEFINITION — start here, no prior knowledge assumed

Each position's vector is shifted to mean 0 and divided by its spread, using only its own numbers.

REAL-WORLD SCENARIO

Marking each student's exam on their own curve: their scores relative to their own average.

MATHEMATICS

μ = *mean of the d numbers,* σ = *their spread,* $y = (x - \mu) / \sqrt{(\sigma^2 + \varepsilon)}$

IMPLEMENTATION

```
def layer_norm(x, g, b, eps=1e-5):
    mu = x.mean(axis=-1, keepdims=True)
    sd = np.sqrt(x.var(axis=-1, keepdims=True) + eps)
    xhat = (x - mu) / sd
    return xhat * g + b, (xhat, sd)
```

ACTUAL OUTPUT

```
 row                     input   mean  spread                            output
   0      [1.0, 2.0, 3.0, 6.0]   3.00  1.8708   -1.0690 -0.5345  0.0000  1.6036
   1  [10.0, 10.0, 10.0, 14.0]  11.00  1.7321   -0.5773 -0.5773 -0.5773  1.7320
largest difference from the formula: 2.22e-16; output means [-0.0, 0.0], spreads [1.0, 1.0]
Actual output from this lesson's run.
```

What it means

The row [1, 2, 3, 6] has mean 3.00 and spread 1.8708; after the norm every row has mean 0 and spread 1. The second row's numbers are about ten times larger, and the other row plays no part in normalising it.

WHY CHOOSE THIS?

Every position is put on the same scale, whatever came before.

WHY NOT THIS? (TRADEOFFS)

It removes the scale information; the learned g and b must put back what is useful.

2. The feed-forward network works on every position on its own

DEFINITION — start here, no prior knowledge assumed

The FFN widens each position's 32 numbers to 128, applies ReLU and narrows them back, with the same weights at every position. Nothing passes between positions: that is attention's job.

REAL-WORLD SCENARIO

After a meeting (attention), everyone goes back to their own desk to think it over (the FFN).

IMPLEMENTATION

```
def ffn(x, P):
    u = x @ P["W1"] + P["c1"]
    r = np.maximum(u, 0.0)
    return r @ P["W2"] + P["c2"], (x, u, r)
def ffn_backward(dy, P, cache):
    x, u, r = cache
    flat = lambda T: T.reshape(-1, T.shape[-1])
    du = (dy @ P["W2"].T) * (u > 0)
    g = dict(W2=flat(r).T @ flat(dy), c2=flat(dy).sum(axis=0), W1=flat(x).T @ flat(du),
        c1=flat(du).sum(axis=0))
    return du @ P["W1"].T, g
```

ACTUAL OUTPUT

```
shuffled: largest |output(shuffled) - shuffled(output)| = 0.00e+00
position 2 alone vs inside the sentence: largest difference = 7.77e-16
weights: W1 32 x 128, W2 128 x 32 -- the same 8352 numbers at every position
Actual output from this lesson's run.
```

What it means

Shuffling positions shuffles the outputs exactly (0.00e+00), and a position fed alone gives its output from inside the sentence (7.77e-16, rounding). The 8352 FFN weights are shared by every position.

WHY CHOOSE THIS?

Per-position thinking with many weights, at no cost in mixing.

WHY NOT THIS? (TRADEOFFS)

On its own it sees nothing of the context.

3. Gradient check

DEFINITION — start here, no prior knowledge assumed

Two stacked post-LN blocks, two pre-LN blocks, and one block with each part removed, through the whole tagger.

REAL-WORLD SCENARIO

Checking every circuit of the house, and each one with a breaker switched off.

MATHEMATICS

$s = x + f(x) \implies ds/dx = I + df/dx$ *(the residual's identity path)*

the loss is computed as a log-softmax: $\log p_k = z_k - \log \Sigma \exp z$

SPECIFICATION

Data	8 words, 3 tags, size 8, 2 heads, FFN 12, three padded sentences; every entry nudged by ±1e-5; Day 62's pass rule.

Why this test

The block's backward pass has many paths; the ablations each remove one.

IMPLEMENTATION

```
def backward(self, dy):
    P, h, c = self.P, self.h, self.c
    g = {k: np.zeros_like(v) for k, v in P.items()}
    flat = lambda T: T.reshape(-1, T.shape[-1])
    if self.use_ffn:
        ds2 = dy if self.pre else self._norm_back(dy, "2", c["n2"], g)
        df_in, gf = ffn_backward(ds2, P, c["ffn"])
        g.update(gf)
        if self.pre:
            df_in = self._norm_back(df_in, "2", c["n2"], g)
        dh1 = df_in + (ds2 if self.res else 0.0)
    else:
        dh1 = dy
    ds1 = dh1 if self.pre else self._norm_back(dh1, "1", c["n1"], g)
    g["Wo"] = flat(c["Z"]).T @ flat(ds1)
    dQ, dK, dV = scaled_dot_product_attention_backward(split_heads(ds1 @ P["Wo"].T, h), c["Q"],
        c["K"], c["V"], c["A"])
    dQ, dK, dV = merge_heads(dQ, h), merge_heads(dK, h), merge_heads(dV, h)
    g["Wq"], g["Wk"], g["Wv"] = flat(c["Xqk"]).T @ flat(dQ), flat(c["Xqk"]).T @ flat(dK),
        flat(c["a_in"]).T @ flat(dV)
    da_in = dQ @ P["Wq"].T + dK @ P["Wk"].T + dV @ P["Wv"].T
    if self.pre:
        da_in = self._norm_back(da_in, "1", c["n1"], g)
    return da_in + (ds1 if self.res else 0.0), g
```

ACTUAL OUTPUT

```
2 blocks, post-LN     : largest |numeric - analytic| = 5.79e-09; every entry passes: True
2 blocks, pre-LN      : largest |numeric - analytic| = 2.57e-09; every entry passes: True
1 block, no residual  : largest |numeric - analytic| = 4.35e-10; every entry passes: True
1 block, no norm      : largest |numeric - analytic| = 2.30e-11; every entry passes: True
1 block, no FFN       : largest |numeric - analytic| = 2.44e-09; every entry passes: True
Actual output from this lesson's run.
```

What it means

Every entry passes in all five versions.

WHAT WAS CHECKED

A bug in the check, found while building this lesson. With Day 62's loss, which takes log(p + 1e-9), one gradient entry failed the pass rule. The analytic gradient was right: the +1e-9 makes the loss a slightly different function from the one the gradient belongs to, and on a small gradient made of large cancelling terms that difference shows. Computing the loss as a log-softmax, which never takes the log of a probability, removes the +1e-9 and the mismatch. That development check is not part of this script's output.

WHY CHOOSE THIS?

All later comparisons rest on checked gradients.

WHY NOT THIS? (TRADEOFFS)

Tiny models; the trained ones run the same code.

4. What keeps the gradient alive

DEFINITION — start here, no prior knowledge assumed

The size of the gradient that reaches the word vectors at the bottom of 1 to 8 untrained blocks, when a gradient of size 1 per word arrives at the top. Five versions: post-LN and pre-LN with residuals, post-LN without them, and no LayerNorm with and without residuals.

REAL-WORLD SCENARIO

Passing a message down a long line of people: does it arrive as a whisper, a shout, or at normal volume?

SPECIFICATION

Data	32 real training sentences, seed 0, untrained blocks; the same top gradient for every version.

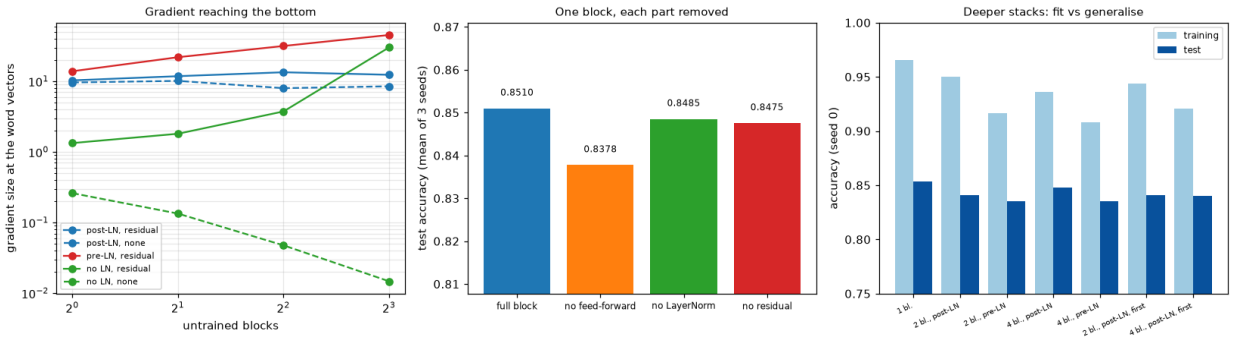
Why this test

Residuals and normalisation are usually explained by how they help gradients through depth; this measures it.

ACTUAL OUTPUT

blocks	post-LN, residual	post-LN, none	pre-LN, residual	no LN, residual	no LN, none
1	1.031e+01	9.631e+00	1.389e+01	1.335e+00	2.607e-01
2	1.182e+01	1.014e+01	2.193e+01	1.809e+00	1.339e-01
4	1.343e+01	7.997e+00	3.164e+01	3.746e+00	4.752e-02
8	1.237e+01	8.498e+00	4.529e+01	3.022e+01	1.472e-02

```
Actual output from this lesson's run.
```



Left: gradient size at the bottom by depth (Section 4). Middle: one block with each part removed (Section 6). Right: deeper stacks, training and test accuracy (Section 7).

What it means

Without either, the gradient fades: from 2.607e-01 at one block to 1.472e-02 at eight. **Residuals without LayerNorm make it grow:** to 3.022e+01 at eight, probably because every block adds its own gradient to the one passing straight through. **With LayerNorm it stays near the same size** with or without residuals (post-LN 1.237e+01 and 8.498e+00 at eight). Pre-LN grows steadily (4.529e+01), probably because its residual path is never normalised.

So in these untrained stacks LayerNorm, not the residual, keeps the size steady. The residual's job shows up elsewhere: it gives every block a direct path, which this size measurement does not capture.

WHY CHOOSE THIS?

A direct measurement, no training noise.

WHY NOT THIS? (TRADEOFFS)

Untrained blocks only; one seed; size is not direction.

5. One block on real tagging

DEFINITION — start here, no prior knowledge assumed

Day 77's tagger rebuilt as one encoder block plus an output layer: the FFN replaces Day 77's hidden layer. Learning rate chosen on validation, then three seeds.

REAL-WORLD SCENARIO

Replacing a home-made gadget with the standard part, then testing it on the same job.

SPECIFICATION

Model	Embedding 32, 4 heads, FFN 128, post-LN, Day 77's encoding (× 3, queries and keys only).
Training	SGD, clip 1.0, batches of 32, 25 epochs, best epoch on validation; lr from 0.5, 1, 2.

Why this test

Does the standard block do at least as well as Day 77's hand-made layer?

IMPLEMENTATION

```
def forward(self, x, pe, kmask):
    P, h, c = self.P, self.h, {}
    a_in, c["n1"] = self._norm(x, "1") if self.pre else (x, None)
    Xqk = a_in + pe
    Q, K, V = split_heads(Xqk @ P["Wq"], h), split_heads(Xqk @ P["Wk"], h), split_heads(a_in @
        P["Wv"], h)
    Zh, A = scaled_dot_product_attention(Q, K, V, kmask)
    Z = merge_heads(Zh, h)
    s1 = Z @ P["Wo"] + (x if self.res else 0.0)
    h1, n1 = (s1, None) if self.pre else self._norm(s1, "1")
    c["n1"] = c["n1"] if self.pre else n1
    c.update(a_in=a_in, Xqk=Xqk, Q=Q, K=K, V=V, A=A, Z=Z)
    if not self.use_ffn:
        self.c = c
        return h1
    f_in, c["n2"] = self._norm(h1, "2") if self.pre else (h1, None)
    fo, c["ffn"] = ffn(f_in, P)
    s2 = fo + (h1 if self.res else 0.0)
    y, n2 = (s2, None) if self.pre else self._norm(s2, "2")
    c["n2"] = c["n2"] if self.pre else n2
    self.c = c
    return y
```

ACTUAL OUTPUT

```
validation accuracy, seed 0: lr 0.5: 0.8428  lr 1.0: 0.8444  lr 2.0: 0.8475  -> lr 2.0
test accuracy: 0.8535  0.8502  0.8492   mean 0.8510   (training accuracy 0.9542)
Day 77's single attention layer + hidden layer (same encoding): 0.8487
Actual output from this lesson's run.
```

What it means

The largest rate tried, lr 2.0, was best on validation, so a larger one might be better still; it was not tried. One block reaches 0.8510 over three seeds, a little above Day 77's 0.8487, with a training accuracy of 0.9542: a large gap, so it fits the training sentences far better than new ones.

WHY CHOOSE THIS?

The standard block works as a drop-in replacement.

WHY NOT THIS? (TRADEOFFS)

A small gain, of the size of the seed spread.

6. Each part removed

DEFINITION — start here, no prior knowledge assumed

The same one-block tagger without the FFN, without LayerNorm, and without the residuals; three seeds each.

REAL-WORLD SCENARIO

Taking parts out of an engine one at a time to see which ones it really needs.

ACTUAL OUTPUT

```
model                seed 0   seed 1   seed 2     mean    train
full block           0.8535   0.8502   0.8492   0.8510   0.9542
no feed-forward      0.8433   0.8334   0.8365   0.8378   0.9239
no LayerNorm         0.8449   0.8464   0.8542   0.8485   0.9491
no residual          0.8499   0.8508   0.8419   0.8475   0.9454
Actual output from this lesson's run.
```

What it means

The FFN matters most: without it, 0.8378 (and training accuracy 0.9239), about 1.3 points down. Without LayerNorm (0.8485) or without the residuals (0.8475) the loss is small, and their seeds overlap the full block's (no LayerNorm 0.8449 to 0.8542; full block 0.8492 to 0.8535). With one block there is little depth for them to help with; Section 4 suggests where they matter.

WHY CHOOSE THIS?

Shows which part earns its place at this size.

WHY NOT THIS? (TRADEOFFS)

One block, one task; three seeds cannot separate differences this small.

7. Deeper: 2 and 4 blocks

DEFINITION — start here, no prior knowledge assumed

Stacks of 2 and 4 blocks, post-LN and pre-LN, with Day 77's encoding added in every block or only in the first; seed 0, the same learning rate.

REAL-WORLD SCENARIO

Adding more floors to a building: worth it only if the foundations and the lift keep up.

Why this test

Depth is what blocks are for. Training accuracy at the best epoch separates a model that cannot fit from one that fits but does not generalise.

ACTUAL OUTPUT

```
model                       positions      val     test    train  best epoch
1 block                         every   0.8475   0.8535   0.9652          19
2 blocks, post-LN               every   0.8307   0.8410   0.9501          22
2 blocks, pre-LN                every   0.8254   0.8349   0.9168          25
4 blocks, post-LN               every   0.8488   0.8478   0.9363          21
4 blocks, pre-LN                every   0.8218   0.8353   0.9082          22
2 blocks, post-LN               first   0.8327   0.8405   0.9438          17
4 blocks, post-LN               first   0.8309   0.8402   0.9204          21
Actual output from this lesson's run.
```

What it means

No deeper stack beat one block on test (0.8535 for seed 0): the best was 4 post-LN blocks at 0.8478; 2 post-LN blocks reached 0.8410, and pre-LN 0.8349 and 0.8353.

The deeper models fit the training data less, not more (training accuracy 0.9501 and 0.9363 post-LN, 0.9168 and 0.9082 pre-LN, against 0.9652 for one block). So they are not overfitting; with plain SGD, a fixed rate and 25 epochs they train less well. Real Transformers are trained with Adam and a warm-up of the learning rate, which this lesson does not use. Adding the encoding only in the first block made no difference at this rate (0.8405 against 0.8410).

This matches Day 68, where a second LSTM layer did not help on the same data.

WHY CHOOSE THIS?

Shows honestly that depth is not free.

WHY NOT THIS? (TRADEOFFS)

Seed 0 only; one learning rate; plain SGD.

Interview preparation — technical questions

Q: Write a post-LN encoder block.

A: h = LN(x + MHA(x)); y = LN(h + FFN(h)), FFN(h) = ReLU(h W1 + c1) W2 + c2.

Q: Pre-LN vs post-LN?

A: Pre-LN normalises before each part and keeps an un-normalised residual path; post-LN normalises after adding.

Q: What does LayerNorm normalise over?

A: The d features of one position, on their own.

Q: Does the FFN mix positions?

A: No: shuffled positions give shuffled outputs exactly (0.00e+00).

Q: Which part mattered most in one block here?

A: The FFN: 0.8378 without it against 0.8510.

Interview preparation — theoretical questions

Q: Why residual connections?

A: They give each block a direct path for the input and the gradient; without LayerNorm, here the gradient faded to 1.472e-02 through 8 blocks without them.

Q: Why did deeper stacks do worse here?

A: They reached lower training accuracy too (0.9082 to 0.9501), so it is training, not overfitting; plain SGD without warm-up.

Q: Why compute the loss as a log-softmax?

A: It is exact and stable; log(p + 1e-9) changes the function and made one gradient check entry fail.

Q: What does the FFN add that attention lacks?

A: Per-position computation with many weights; attention only averages values.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> An encoder block is attention and a position-wise FFN, each wrapped in add & norm. One block tags Week 9's sentences at 0.8510. Its FFN is the part that mattered; LayerNorm kept untrained gradients the same size through 8 blocks; and deeper stacks trained less well under plain SGD, so depth did not help here.

- One block: 0.8510 (three seeds), training 0.9542.
- Without the FFN 0.8378; without LayerNorm 0.8485; without residuals 0.8475.
- Untrained, 8 blocks: no norm and no residual 1.472e-02; residual only 3.022e+01; post-LN 1.237e+01.
- Deeper (seed 0): 2 post-LN 0.8410, 4 post-LN 0.8478, 2 pre-LN 0.8349, 4 pre-LN 0.8353, against 0.8535.

Suggested watch order, if starting from zero

- 1. Encoder Architecture in Transformers, Step by Step Guide (Learn With Jay) — the block, step by step.
- 2. PostLN, PreLN and ResiDual Transformers (Machine Learning Studio) — where the norm goes, and why it matters for depth.

Day 79 preview

- **LayerNorm vs BatchNorm.** Day 54 used batch normalisation in a CNN. Day 79 puts it inside today's block, on padded sentences of different lengths, and compares it with LayerNorm.
- Today's exercise script, `day78_exercise.py`, is one encoder block, forward and backward, gradient-checked: the second part of Day 83's Transformer.